

MATLAB

COMPLETE NOTES

Easy Language • Simple Examples • Important Topics Highlighted

★ Red star = Important Topic (read carefully, exam / interview favorite)

MMOC TEACH
MADE BY PRAVENDRA

INDEX

★ 1. Introduction to MATLAB

★ 2. Variables and Matrices

- 3. Basic Operators

★ 4. Vectors and Matrix Operations

- 5. Conditional Statements (if, elseif, else)
- 6. Loops - for, while

★ 7. Functions

★ 8. Plotting Graphs

- 9. String Handling
- 10. Cell Arrays
- 11. File Input and Output

★ 12. Basic Linear Algebra Operations

★ 1. Introduction to MATLAB [IMPORTANT TOPIC]

Explanation:

MATLAB (Matrix Laboratory) is a programming environment designed specially for maths, engineering, and scientific calculations. It works especially well with matrices (grids of numbers), which is where its name comes from.

Key Points:

- Widely used by engineers, scientists, and students for calculations and simulations.
- Excellent built-in support for graphs and visual plots.
- Everything in MATLAB is treated as a matrix, even a single number.

Example Code:

```
disp('Hello, World!')
```

Output:

```
Hello, World!
```

 *Note: disp() displays a value without extra labels; typing a variable name alone (without a semicolon) also shows its value with a label.*

★ 2. Variables and Matrices *[IMPORTANT TOPIC]*

Explanation:

A variable stores a value. In MATLAB, even a simple number is technically stored as a 1x1 matrix. Larger matrices are created using square brackets [].

Example Code:

```
age = 21;  
name = 'Pravendra';  
  
matrix = [1 2 3; 4 5 6];    % a 2x3 matrix  
disp(matrix)
```

Output:

```
1    2    3  
4    5    6
```

 *Note: A semicolon (;) at the end of a line hides the output for that line; without it, MATLAB automatically displays the result.*

3. Basic Operators

Explanation:

Operators perform actions on values, such as maths or comparisons.

Example Code:

```
a = 10;  
b = 3;  
disp(a + b)    % 13  
disp(mod(a, b)) % 1 (remainder)
```

Output:

```
13  
1
```

★ 4. Vectors and Matrix Operations [IMPORTANT TOPIC]

Explanation:

MATLAB is built around working with vectors (a single row or column of numbers) and matrices (a full grid of numbers), and it can perform maths on ALL their values at once, without needing a loop.

Example Code:

```
v = [10 20 30];  
disp(v * 2)      % multiplies every value by 2 at once  
  
A = [1 2; 3 4];  
B = [5 6; 7 8];  
disp(A + B)      % adds matrices together
```

Output:

```
20    40    60  
6      8  
10    12
```

 *Note: This ability to do maths on a whole vector/matrix at once (instead of looping through each value) is called 'vectorization' — it makes MATLAB code both shorter and faster.*

5. Conditional Statements (if, elseif, else)

Explanation:

Conditional statements let the program make decisions based on whether something is true or false.

Example Code:

```
marks = 75;

if marks >= 90
    disp('Grade A')
elseif marks >= 60
    disp('Grade B')
else
    disp('Grade C')
end
```

Output:
Grade B

 *Note: Every if, for, and while block in MATLAB must be closed with the word 'end'.*

6. Loops - for, while

Explanation:

Loops repeat a block of code multiple times. A MATLAB 'for' loop commonly uses a range like 1:3 to repeat a fixed number of times.

Example Code:

```
for i = 1:3
    disp(['Count: ', num2str(i)])
end
```

Output:

```
Count: 1
Count: 2
Count: 3
```

 *Note: num2str() converts a number into text, so it can be joined with other text using square brackets.*

★ 7. Functions *[IMPORTANT TOPIC]*

Explanation:

A function in MATLAB is usually written in its own file (with the same name as the function), starting with the word 'function'.

Example Code:

```
function total = addNumbers(a, b)
    total = a + b;
end

% calling it from another script:
result = addNumbers(5, 7);
disp(result)
```

Output:

12

★ 8. Plotting Graphs [IMPORTANT TOPIC]

Explanation:

MATLAB is extremely well known for its powerful and easy-to-use graph plotting tools, widely used to visualize data, functions, and simulation results.

Example Code:

```
x = 1:10;  
y = x.^2;    % squares every value in x  
  
plot(x, y)  
title('Graph of y = x^2')  
xlabel('x values')  
ylabel('y values')
```

Output:

(Shows a curved line graph of $y = x^2$, with labeled axes and a title.)

 *Note: The `.^` operator squares EACH value in the vector separately; a plain `^` would try to do full matrix power, which needs a square matrix.*

9. String Handling

Explanation:

MATLAB can store text using single-quoted 'char arrays' or double-quoted "strings" (a more modern type). Various built-in functions help work with text.

Example Code:

```
name = "Pravendra";  
greeting = strcat("Hello, ", name);  
disp(greeting)  
disp(upper(name))
```

Output:

```
Hello, Pravendra  
PRAVENDRA
```

10. Cell Arrays

Explanation:

A normal MATLAB matrix can only hold ONE data type. A cell array can hold DIFFERENT types of data together — like numbers, text, and matrices, all in one variable — similar to a general-purpose list.

Example Code:

```
myCell = {"Pravendra", 21, [1 2 3]};  
disp(myCell{1})    % curly braces {} access cell contents  
disp(myCell{2})
```

Output:

```
Pravendra  
21
```

11. File Input and Output

Explanation:

MATLAB can save data to a file, or load data from a file, which is useful for storing results or importing datasets for analysis.

Example Code:

```
data = [1 2 3 4 5];  
save('mydata.mat', 'data'); % saves the variable to a file  
  
loadedData = load('mydata.mat');  
disp(loadedData.data)
```

Output:

```
1      2      3      4      5
```

★ 12. Basic Linear Algebra Operations *[IMPORTANT TOPIC]*

Explanation:

Since MATLAB was built around matrices, it comes with very strong built-in support for linear algebra — a branch of maths dealing with matrices and vectors, widely used in engineering and machine learning.

Example Code:

```
A = [1 2; 3 4];  
  
disp(transpose(A))    % flips rows and columns  
disp(inv(A))          % finds the inverse of the matrix  
disp(det(A))          % finds the determinant
```

Output:

```
1    3  
2    4  
-2.0000    1.0000  
1.5000   -0.5000  
-2
```